

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

FitTracker

Memoria Técnica de Arquitectura, Especificación SRS y Modelo Matemático de Sobrecarga Progresiva Automática

Lead Software Architect & Tech Lead

14 de agosto de 2026

Resumen

El presente documento constituye la especificación técnica y de ingeniería de software para el desarrollo del sistema móvil **FitTracker**. Diseñado bajo un enfoque *Local-First* y arquitectura modular *Clean Architecture*, FitTracker automatiza la gestión de entrenamientos de fuerza mediante un motor de sobrecarga progresiva basado en formulación matemática de estimación de 1RM (*1RepetitionMaximum*) y autorregulación por RPE/RIR. Se detalla la especificación de requerimientos (SRS), casos de uso UML, diseño de base de datos SQLite y el modelo algorítmico de progresión.

Índice

1. Introducción y Objetivos	3
1.1. Contexto y Justificación	3
1.2. Objetivos del Sistema	3
1.2.1. Objetivo General	3
1.2.2. Objetivos Específicos	3
1.3. Agradecimientos y Fuentes de Datos	3
2. Especificación de Requerimientos de Software (SRS)	4
2.1. Requerimientos Funcionales (RF)	4
2.2. Requerimientos No Funcionales (RNF)	4
3. Casos de Uso UML	5
3.1. Actores del Sistema	5
3.2. Especificación Detallada de Caso de Uso CU-01	5
4. Diseño de Arquitectura y Base de Datos	6
4.1. Arquitectura del Sistema (Clean Architecture)	6
4.2. Modelo Entidad-Relación y Esquema SQLite	6
5. Algoritmo de Sobrecarga Progresiva Automática	7
5.1. Formulación Matemática de Estimación de 1RM	7
5.2. Volumen Efectivo e Incremento Progresivo Automático	7
5.3. Regla de Incremento para la Siguiente Sesión	7

1. Introducción y Objetivos

1.1. Contexto y Justificación

En la disciplina del entrenamiento de fuerza y la hipertrofia muscular, el principio de **sobrecarga progresiva** constituye el pilar fundamental para desencadenar adaptaciones fisiológicas a largo plazo. No obstante, la inmensa mayoría de las aplicaciones móviles del mercado actual sufren de dos deficiencias críticas:

1. Dependencia estricta de conectividad a Internet y servidores en la nube (*Cloud-First*), lo que degrada la experiencia de usuario en gimnasios con baja cobertura o zonas subterráneas.
2. Falta de algoritmos deterministas de sobrecarga progresiva automática basados en variables fisiológicas y métricas cuantitativas como el 1RM estimado y la escala RPE/RIR (*Rating of Perceived Exertion / Repetitions in Reserve*).

FitTracker nace como un sistema móvil orientado a solucionar ambas limitaciones mediante una arquitectura **Local-First** impulsada por una base de datos SQLite integrada y un motor algorítmico adaptativo de progresión.

1.2. Objetivos del Sistema

1.2.1. Objetivo General

Diseñar, construir e implementar una aplicación móvil multiplataforma (React Native + Expo en TypeScript) caracterizada por un funcionamiento 100 % offline y una arquitectura limpia (*Clean Architecture*), capaz de calcular e incrementar automáticamente la carga y volumen de trabajo de los atletas según su rendimiento histórico.

1.2.2. Objetivos Específicos

- Implementar una capa de persistencia ultrarrápida local mediante SQLite, garantizando tolerancia a fallos de red y cero latencia en el registro de series.
- Desarrollar el módulo matemático de estimación de 1RM (*1RepetitionMaximum*) promediando los modelos clásicos de Epley y Brzycki con corrección por RIR.
- Proveer un sistema de gestión modular separado estrictamente en capas de Dominio, Casos de Uso, Repositorios y UI.
- Proporcionar una memoria técnica formal para auditoría e investigación académica.

1.3. Agradecimientos y Fuentes de Datos

Agradecemos formalmente al desarrollador **Hasan Yıldırım** por la creación y mantenimiento del repositorio *Open Source exercises-dataset*. Su dataset estructurado en JSON proporciona la base de datos completa de ejercicios, metadatos y elementos multimedia que nutren el sistema de siembra (*seeding*) local offline en nuestra arquitectura de persistencia.

2. Especificación de Requerimientos de Software (SRS)

La presente especificación sigue la adaptación del estándar IEEE 830 para sistemas móviles offline-first.

2.1. Requerimientos Funcionales (RF)

Tabla 1: Requerimientos Funcionales del Sistema

Código	Nombre	Descripción
RF-01	Gestión de Catálogo de Ejercicios	El sistema permitirá crear, modificar y categorizar ejercicios por grupo muscular principal y secundario.
RF-02	Configuración de Rutinas	El sistema permitirá definir rutinas compuestas por bloques de ejercicios, rango de reps objetivo y RPE deseado.
RF-03	Registro de Series en Vivo	Permite registrar peso (kg), repeticiones (r) y RPE/-RIR (R) en tiempo real con temporizador de descanso automatizado.
RF-04	Cálculo de Sobrecarga Autómata	El sistema calculará al finalizar la rutina las sugerencias de carga/reps para la siguiente sesión del ejercicio.
RF-05	Estimación de 1RM	El sistema calculará el 1RM histórico y actual por sesión mediante fórmulas validadas.
RF-06	Registro Nutricional Offline	Registro diario de macronutrientes (proteínas, carbohidratos, grasas) y calorías totales sin requerir API externa.
RF-07	Métricas Corporales	Registro de peso corporal, porcentaje de grasa y circunferencia de pliegues.

2.2. Requerimientos No Funcionales (RNF)

Tabla 2: Requerimientos No Funcionales del Sistema

Código	Nombre	Métrica / Criterio de Aceptación
RNF-01	Operatividad Offline	El 100 % de las operaciones de lectura y escritura deben ejecutarse localmente sin conexión a red.
RNF-02	Rendimiento de Escritura	La inserción de una serie completada en la DB debe tomar menos de 50 ms .
RNF-03	Portabilidad	El código base en React Native (TypeScript) debe ser portable entre Android y iOS sin cambios de lógica.
RNF-04	Integridad de Datos	La base de datos SQLite debe utilizar transacciones ACID para evitar corrupción ante cierres inesperados.

3. Casos de Uso UML

3.1. Actores del Sistema

- **Atleta / Usuario Final:** Persona que ejecuta la rutina de entrenamiento, registra series y consulta las métricas de sobrecarga.

3.2. Especificación Detallada de Caso de Uso CU-01

Tabla 3: Caso de Uso CU-01: Registrar Serie de Entrenamiento

Caso de Uso	CU-01: Registrar Serie de Entrenamiento y Calcular Progresión
Actor Principal	Atleta / Usuario Final
Precondiciones	Existe al menos un ejercicio en el catálogo y una rutina activa.
Flujo Principal	1. El usuario selecciona el ejercicio actual en la sesión activa. 2. El sistema muestra la sugerencia de peso y reps calculada previamente. 3. El usuario ingresa la carga efectuada (<i>kg</i>), reps ejecutadas y RPE (6,0 – 10,0). 4. El usuario presiona el botón "Completar Serie". 5. El sistema persiste la serie en SQLite, inicia el cronómetro de descanso e incrementa el indicador de series. 6. El motor de sobrecarga actualiza el 1RM estimado de la sesión.
Flujo Alternativo	<i>3a. El usuario marca la serie como "Serie de Calentamiento":</i> - El sistema guarda la serie pero la excluye de los cálculos de volumen efectivo y 1RM récord. <i>3b. El usuario indica fallo muscular (RPE 10 / RIR 0):</i> - El sistema ajusta el factor de fatiga acumulada para el cálculo de la siguiente sesión.
Postcondiciones	Serie guardada en SQLite; el temporizador de descanso emite notificación sonora/vibración.

4. Diseño de Arquitectura y Base de Datos

4.1. Arquitectura del Sistema (Clean Architecture)

FitTracker implementa la arquitectura limpia (*Clean Architecture*) adaptada a módulos funcionales (*Feature-First*).

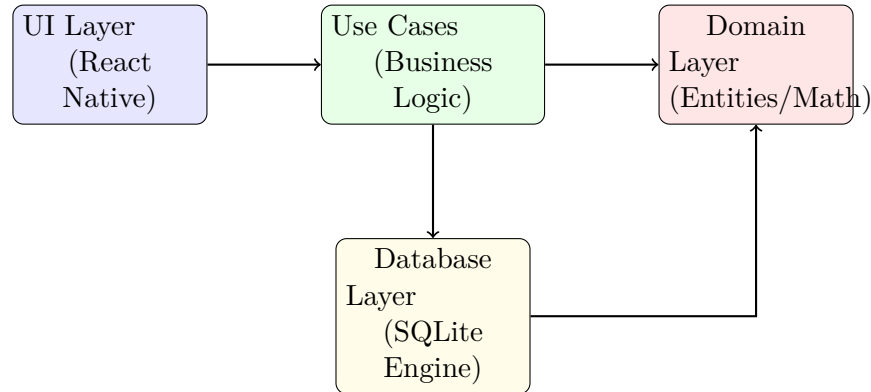


Figura 1: Diagrama de Dependencias de Capas en Clean Architecture.

4.2. Modelo Entidad-Relación y Esquema SQLite

```
1 -- Tabla de Ejercicios
2 CREATE TABLE exercises (
3     id TEXT PRIMARY KEY NOT NULL,
4     name TEXT NOT NULL,
5     category TEXT NOT NULL, -- Chest, Back, Legs, etc.
6     exercise_type TEXT NOT NULL, -- Compound vs Isolation
7     created_at INTEGER NOT NULL
8 );
9
10 -- Tabla de Sesiones de Entrenamiento
11 CREATE TABLE workout_sessions (
12     id TEXT PRIMARY KEY NOT NULL,
13     name TEXT NOT NULL,
14     date INTEGER NOT NULL,
15     notes TEXT
16 );
17
18 -- Tabla de Series Registradas
19 CREATE TABLE exercise_sets (
20     id TEXT PRIMARY KEY NOT NULL,
21     session_id TEXT NOT NULL,
22     exercise_id TEXT NOT NULL,
23     set_order INTEGER NOT NULL,
24     weight_kg REAL NOT NULL,
25     reps INTEGER NOT NULL,
26     rpe REAL NOT NULL,
27     is_warmup INTEGER NOT NULL DEFAULT 0,
28     estimated_1rm REAL NOT NULL,
29     FOREIGN KEY(session_id) REFERENCES workout_sessions(id) ON DELETE CASCADE,
30     FOREIGN KEY(exercise_id) REFERENCES exercises(id) ON DELETE CASCADE
31 );
```

Listing 1: DDL del Esquema Relacional SQLite

5. Algoritmo de Sobrecarga Progresiva Automática

5.1. Formulación Matemática de Estimación de 1RM

Para cuantificar la fuerza relativa en una serie de r repeticiones ejecutadas con carga w (expresada en kg) a una percepción de esfuerzo $RPE \in [6,0,10,0]$, calculamos en primer lugar las repeticiones en recámara equivalentes (*Repetitions in Reserve, RIR*):

$$RIR = 10,0 - RPE \quad (1)$$

Las repeticiones estimadas a fallo absoluto $r_{\max}$ se determinan como:

$$r_{\max} = r + RIR \quad (2)$$

Utilizamos una combinación ponderada de los modelos no lineales de **Epley** y **Brzycki**:

1. Fórmula de Epley:

$$1RM_{\text{Epley}} = w \cdot \left(1 + \frac{r_{\max}}{30} \right) \quad (3)$$

2. Fórmula de Brzycki:

$$1RM_{\text{Brzycki}} = w \cdot \frac{36}{37 - r_{\max}} \quad (4)$$

El 1RM estimado final de la serie $1RM_{\text{est}}$ se obtiene promediando ambos modelos:

$$1RM_{\text{est}} = \frac{1RM_{\text{Epley}} + 1RM_{\text{Brzycki}}}{2} \quad (5)$$

5.2. Volumen Efectivo e Incremento Progresivo Automático

El **Volumen Efectivo Total** (V_{efectivo}) de una sesión para un ejercicio dado, considerando únicamente series efectivas ($is_warmup = 0$), se expresa como:

$$V_{\text{efectivo}} = \sum_{i=1}^N w_i \cdot r_i \cdot \alpha(RPE_i) \quad (6)$$

donde $\alpha(RPE_i)$ es el coeficiente de ponderación de fatiga:

$$\alpha(RPE) = \begin{cases} 1,0 & \text{si } RPE \geq 8,0 \\ 0,85 & \text{si } 7,0 \leq RPE < 8,0 \\ 0,70 & \text{si } RPE < 7,0 \end{cases} \quad (7)$$

5.3. Regla de Incremento para la Siguiente Sesión

La recomendación de carga w_{next} para la subsiguiente sesión se ajusta según la media de RIR_{prom} de la sesión actual y el logro del límite superior del rango de repeticiones objetivo $[R_{\min}, R_{\max}]$:

$$w_{\text{next}} = \begin{cases} w_{\text{actual}} + \Delta w & \text{si } r_i \geq R_{\max} \forall i \text{ y } RIR_{\text{prom}} \geq 1,5 \\ w_{\text{actual}} & \text{si } R_{\min} \leq r_i < R_{\max} \text{ o } 0,5 \leq RIR_{\text{prom}} < 1,5 \\ w_{\text{actual}} \cdot (1 - \beta) & \text{si } RIR_{\text{prom}} < 0,5 \text{ (deload o fatiga excesiva)} \end{cases} \quad (8)$$

donde Δw es el incremento estándar ($2,5 \text{ kg}$ para ejercicios compuestos, $1,25 \text{ kg}$ para ejercicios de aislamiento) y $\beta \in [0,05, 0,10]$ es el factor de descarga por fatiga acumulada.